

Functions

Programming for Problem Solving (PPS)

GTU # 3110003

USING

{C}

Programming



Prof. Nilesh Gambhava

Computer Engineering Department,
Darshan Institute of Engineering & Technology, Rajkot



What is Function?

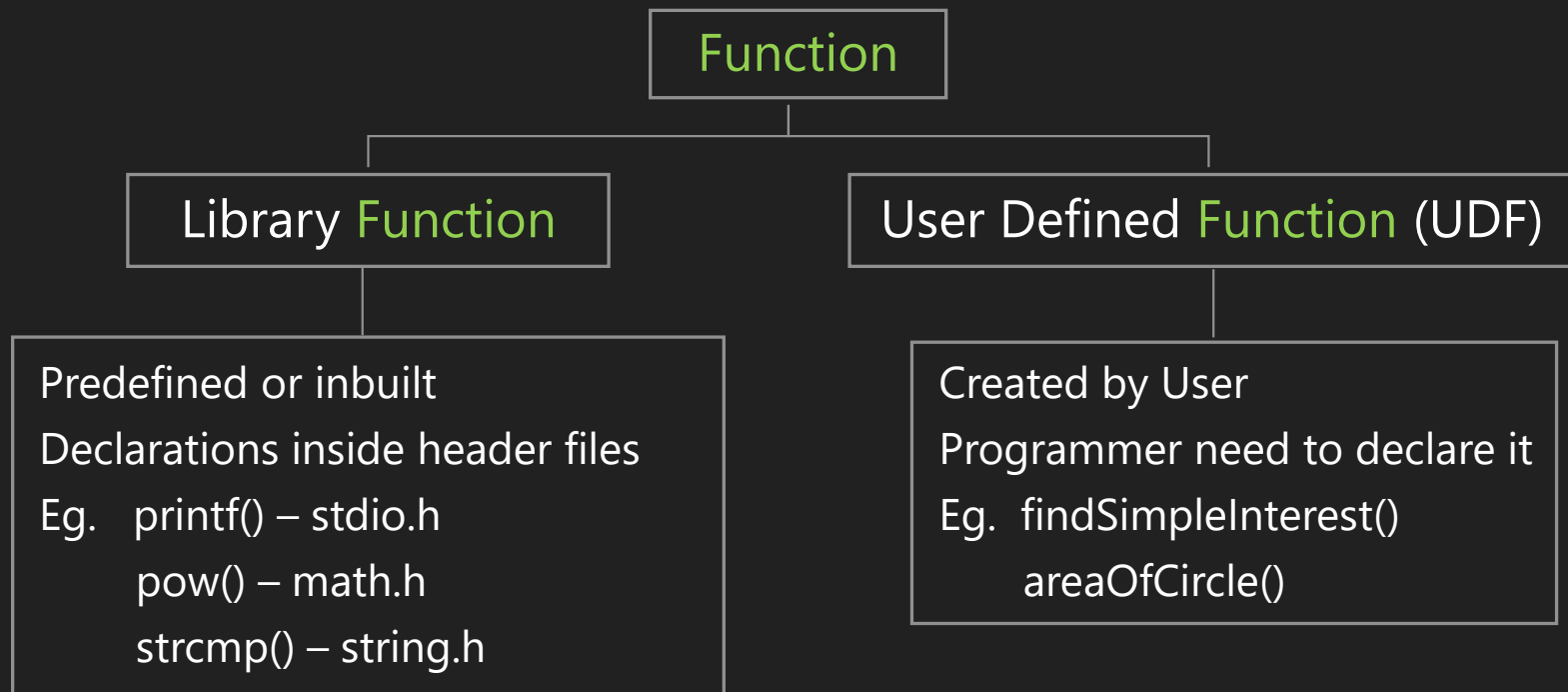
- ▶ A **function** is a group of statements that perform a specific task.
- ▶ It divides a large program into smaller parts.
- ▶ A **function** is something like hiring a person to do a specific job for you.
- ▶ Every C program can be thought of as a collection of these functions.
- ▶ Program execution in C language starts from the main function.

Syntax

```
void main()  
{  
    // body part  
}
```

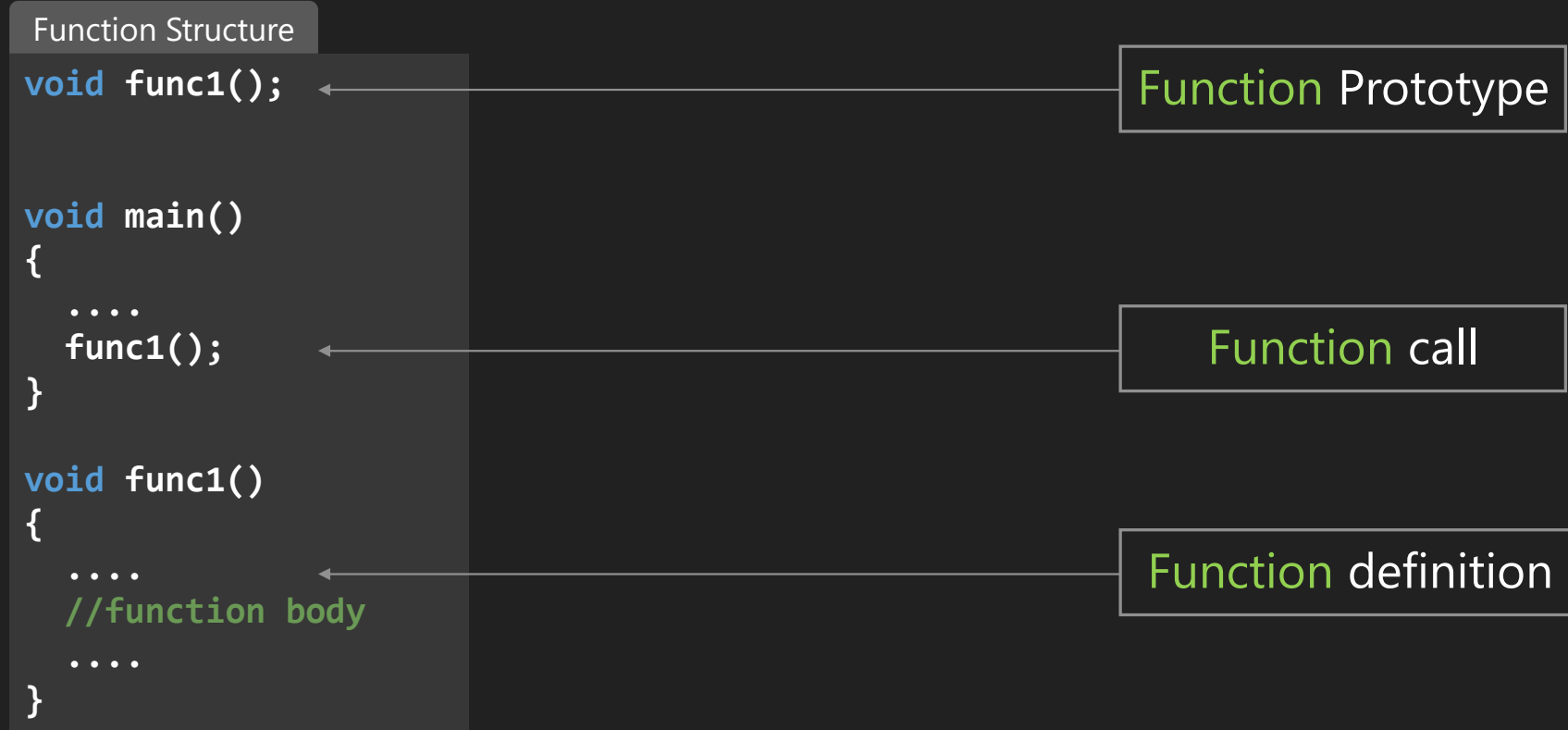
- ▶ Why **function** ?
 - ➔ Avoids rewriting the same code over and over.
 - ➔ Using functions it becomes easier to write programs and keep track of what they doing.

Types of Function



Program Structure for Function

- ▶ When we use a user-defined function program structure is divided into three parts.



Function Prototype and Function Definition

Syntax

Declaration

```
return-type function-name (arg-1, arg 2, ...);
```

Definition

```
return-type function-name (arg-1, arg 2, ...)  
{  
    //... Function body  
}
```

Example

```
void addition(int, int);
```

```
void addition(int x, int y)  
{  
    printf("Addition is=%d", (x+y));  
}
```

Function Prototype

- ▶ A **function** Prototype also know as function declaration.
- ▶ A **function** declaration tells the compiler about a function name and how to call the function.
- ▶ It defines the function before it is being used or called.
- ▶ A **function** prototype needs to be written at the beginning of the program.

Syntax

```
return-type function-name (arg-1, arg 2, ...);
```

Example

```
void addition(int, int);
```

Function Definition

- ▶ A **function** definition defines the functions header and body.
- ▶ A **function** header part should be identical to the function prototype.
 - ➔ Function return type
 - ➔ Function name
 - ➔ List of parameters
- ▶ A **function** body part defines function logic.
 - ➔ Function statements

Syntax

```
return-type function-name (arg-1, arg 2, ...)  
{  
    //... Function body  
}
```

Example

```
void addition(int x, int y)  
{  
    printf("Addition is=%d", (x+y));  
}
```

WAP to add two number using add(int, int) Function

Program

```
1  #include <stdio.h>
2  void add(int, int); // function declaration
3
4  void main()
5  {
6      int a = 5, b = 6;
7      add(a, b); // function call
8  }
9
10 void add(int x, int y) // function definition
11 {
12     printf("Addition is = %d", x + y);
13 }
```

Output

Addition is = 11

Actual parameters and Formal parameters

- ▶ Values that are passed to the called function from the main function are known as **Actual** parameters.
- ▶ The variables declared in the function prototype or definition are known as **Formal** parameters.
- ▶ When a method is called, the **formal** parameter is temporarily "bound" to the **actual** parameter.

Actual parameters

```
void main()
{
    int a = 5, b = 6;
    add(a, b); // a and b are the
               // actual parameters in this call.
}
```

Formal parameters

```
void add(int x, int y) // x and y are
                       // formal parameters.
{
    printf("Addition is = %d", x + y);
}
```

Return Statement

- ▶ If function is returning a value to calling function, it needs to use the keyword **return**.
- ▶ The called function can only return one value per call.

Syntax

```
return;
```

Or

```
return (expression);
```

WAP to find maximum number from two number

Program

```
1  #include <stdio.h>
2  int max(int a, int b);
3  void main()
4  {
5      int a = 100;
6      int b = 200;
7      int maxvalue;
8      maxvalue = max(a, b);
9      printf("Max value is : %d\n",
10     maxvalue);
11 }
12 int max(int a, int b)
13 {
14     if (a > b)
15         return a; // return a
16     else
17         return b; // return b
18 }
```

Output

Max value is : 200

WAP to calculate the Power of a Number

Program

```
1  #include <stdio.h>
2  int power(int, int);
3  void main()
4  {
5      int num, pow, res;
6      printf("Enter any number : ");
7      scanf("%d", &num);
8      printf("Enter power of number : ");
9      scanf("%d", &pow);
10     res = power(num, pow);
11     printf("%d's power %d = %d", num, pow, res);
12 }
13 int power(int n, int p)
14 {   int r = 1;
15     while (p >= 1)
16     {
17         r = r * n;
18         p--;
19     }
20     return r;}
```

Output

```
Enter any number : 5
Enter power of number : 3
5's power 3 = 125
```

WAP to find Factorial of a Number

Program

```
1  #include <stdio.h>
2  int fact(int);
3  int main()
4  {
5      int n, f;
6      printf("Enter the number :\n");
7      scanf("%d", &n);
8      f = fact(n);
9      printf("factorial = %d", f);
10 }
11 int fact(int n)
12 {
13     int i, fact = 1;
14     for (i = 1; i <= n; i++)
15         fact = fact * i;
16     return fact;
17 }
```

Output

```
Enter the number :
5
factorial = 120
```

WAP to check Number is Prime or not

Program

```
1  #include <stdio.h>
2  int checkPrime(int);
3  void main()
4  {
5      int n1, prime;
6      printf("Enter the number :");
7      scanf("%d", &n1);
8      prime = checkPrime(n1);
9      if (prime == 1)
10         printf("The number %d is a prime\n", n1);
11     else
12         printf("The number %d is not a prime\n", n1);
13 }
```

Program contd.

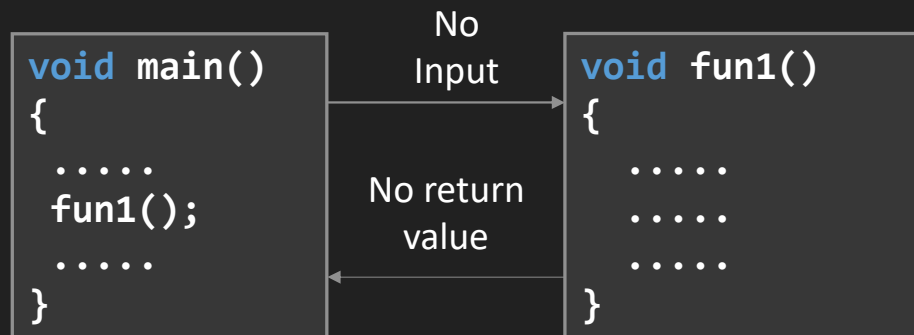
```
14 int checkPrime(int n1)
15 {
16     int i = 2;
17     while (i <= n1 / 2)
18     {
19         if (n1 % i == 0)
20             return 0;
21         else
22             i++;
23     }
24     return 1;
}
```

Output

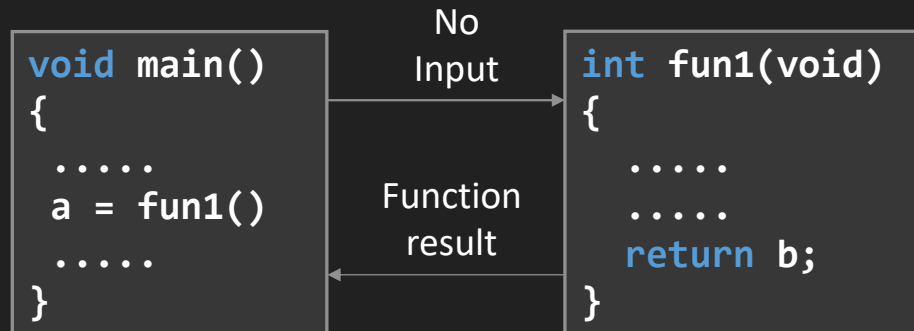
```
Enter the number :7
The number 7 is a prime number.
```

Category of Function

(1) **Function** with no argument and but no return value

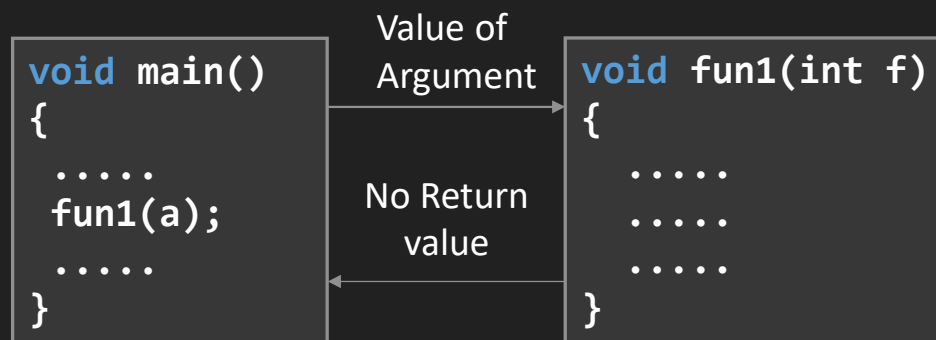


(2) **Function** with no argument and returns value

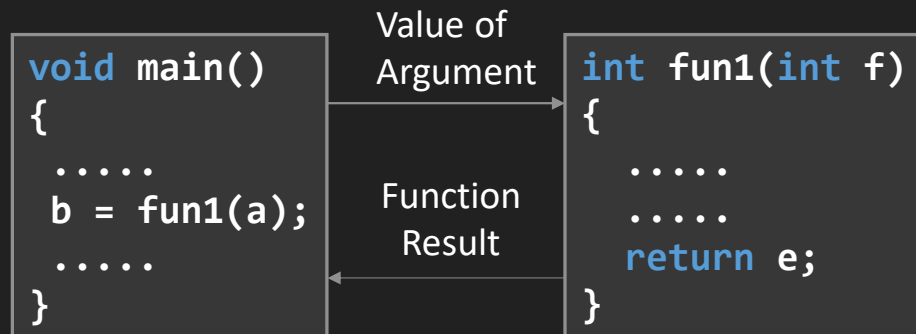


Category of Function cont.

(3) Function with argument and but no return value



(4) Function with argument and returns value



Storage Classes

- ▶ **Storage** class decides the scope, lifetime and memory allocation of variable.
- ▶ Scope of a variable is the boundary within which a variable can be used.

Storage Specifier	Storage	Initial Value	Scope	Life	Example
Automatic { auto }	Stack	Garbage	Within block	End of block	<code>int a; auto int a;</code>
Register { register }	CPU register	Garbage	Within block	End of block	<code>register int var;</code>
External { extern }	Data segment	Zero	Global Multiple file	Till end of program	<code>extern int var;</code>
Static { static }	Data segment	Zero	Within block	Till end of program	<code>static extern int var; static int var;</code>

Static Example

Program

```
1  #include <stdio.h>
2  int incrementCounter();
3
4  void main()
5  {
6      printf("Counter = %d \n", incrementCounter());
7      printf("Counter = %d \n", incrementCounter());
8  }
9
10 int incrementCounter()
11 {
12     static int count = 0; // static variable
13     count++;
14     return count;
15 }
```

Output

```
Counter = 1
Counter = 2
```

Advantages of Function

- ▶ Using **function** we can avoid rewriting the same logic or code again and again in a program.
- ▶ We can track or understand large program easily when it is divide into **functions**.
- ▶ It provides reusability.
- ▶ It help in testing and debugging because it can be tested for errors individually in the easiest way.
- ▶ Reduction in size of program due to code of a **function** can be used again and again, by calling it.

Practice Programs

- 1) WAP to count simple interest using function.
- 2) WAP that defines a function to add first n numbers.
- 3) WAP using global variable, static variable.
- 4) WAP that will scan a character string passed as an argument and convert all lowercase character into their uppercase equivalents.
- 5) Build a function to check number is prime or not. If number is prime then function return value 1 otherwise return 0.
- 6) Write a program to calculate nCr using user defined function. $nCr = n! / (r! * (n-r)!)$
- 7) Create a function to swap the values of two variables.
- 8) Write a function which takes 2 numbers as parameters and returns the gcd of the 2 numbers. Call the function in main().



Thank you

